

٣. التقرير الأساسي (Dot Net Versions) : Task 2) , Namespace, .Net Core, and Solution

Section 1: History & Evolution of .NET Versions

- **.NET Framework (1.0 to 4.8.1) [2002 – 2019]**
 - البيئة الأولى الموجهة بشكل أساسي لنظام Windows.
 - اعتمدت على محرك التشغيل CLR ومرت بتطورات كبيرة مثل تقديم Generics في الإصدار ٢,٠، و LINQ في الإصدار ٣,٥، وميزات async/await في الإصدار ٤,٥.
- **.NET Core (1.0 to 3.1) [2016 – 2019]**
 - إعادة بناء شاملة للمنصة لتصبح **Cross-Platform** (تشغل على Windows, Linux, macOS)، مفرغة من الاعتماد على نظام Windows ومفتوحة المصدر (Open-Source) ومصممة للأداء العالي (Cloud & Microservices).
- **Unified .NET (.NET 5, 6, 7, 8, 9) [2020 – Present]**
 - قامت ميكروسوفت بدمج .NET Framework و NET Core و Xamarin في منصة واحدة موحدة تُدعى **.NET**.
 - تقدم دعماً كبيراً للـ Cross-Platform، وتطوير الويب (ASP.NET Core)، والموبايل (NET MAUI)، والذكاء الاصطناعي، مع إصدارات دعم طويل الأجل (LTS) مثل NET 6 و NET 8.

Section 2: Understanding Namespaces

- **المفهوم:** الـ Namespace هو غلاف تنظيمي (Logical Container) يُستخدم لتجميع الـ Classes و الـ Interfaces و الـ Structs المتعلقة ببعضها تحت اسم واحد.
- **الفائدة:**
 ١. يمنع تضارب الأسماء (Name Collisions)؛ فمثلاً يمكن وجود صفين باسم User إذا كان كل منهما داخل Namespace مختلف (App.Models.User مقابل App.Services.User).
 ٢. تنظيم المشروع وسهولة استدعاء المكتبات باستخدام الكلمة المفتاحية `using`.

مثال برمجي:

```
C#
namespace FinancialApp.Services
{
    public class PaymentProcessor
    {
        public void Process()
        {
            // Payment logic
        }
    }
}
```

Section 3: Deep Dive into .NET Core

- **التعريف:** هي بيئة تشغيل برمجية خفيفة، مفتوحة المصدر، عالية الأداء، وتعمل على مختلف أنظمة التشغيل (Cross-Platform).

- أهم الميزات:
 - **Cross-Platform Support**: تعمل على Windows و Linux و macOS بنفس الكفاءة.
 - **High Performance**: تفوق أداؤها على البيئات القديمة بفضل تحسينات الـ Memory Allocation والـ JIT.
 - **Modular Architecture**: تعتمد على حزم NuGet، مما يعني أن التطبيق يحمل فقط المكتبات التي يحتاجها دون تحميل بيئة التشغيل كاملة.
 - **CLI-Driven**: إمكانية بناء وتشغيل التطبيقات بالكامل عبر السطر البرمجي (`dotnet run` ، `build`).

Section 4: What is a Solution in .NET?

- **التعريف:** الـ **Solution** (ملف بامتداد `.sln`) هو الحاوية الهيكلية الأساسية التي تُستخدم داخل Visual Studio لإدارة مشروع واحد أو عدة مشاريع مرتبطة ببعضها (Projects).
- **العلاقة بين الـ Solution والـ Project:**
 - **Project (ملف .csproj)**: يمثل تطبيقاً محدداً أو مكتبة كود (مثل Web API, Class Library, Unit Tests) ويحتوي على الملفات البرمجية والإعدادات الخاصة بها.
 - **Solution (ملف .sln)**: يحتوي على مشروع واحد أو أكثر، ويحدد كيفية بناء هذه المشاريع معاً والعلاقات/الاعتمادات بينها (Build Order & Project Dependencies).